

Practical 8

Movie Review Sentiment Analysis Project

Name: Rishil Pawar

Roll No.: C1-18

Subject: Natural Language Processing (NLP) Lab

Practical No.: 8

GitHub Repository:

<https://github.com/TheSpectre542005/Movie-Review-Prediction/tree/main>

Table of Contents

1. 1. Introduction / Aim
2. 2. Dataset Description
3. 3. Libraries & Dependencies
4. 4. Step 1: Install & Import Libraries
5. 5. Step 2: Load & Explore Dataset (EDA)
6. 6. Step 3: Text Preprocessing
7. 7. Step 4: Feature Extraction (TF-IDF)
8. 8. Step 5: Model Training & Evaluation
9. 9. Step 6: Comparative Analysis
10. 10. Step 7: Best Model Selection
11. 11. Step 8: Model Saving & Loading
12. 12. Step 9: GUI-based Prediction (Gradio)
13. 13. Conclusion

1. Introduction / Aim

The aim of this practical is to build a complete Movie Review Sentiment Analysis system using Natural Language Processing (NLP) techniques. The project uses the IMDB Dataset containing 50,000 movie reviews (25,000 positive and 25,000 negative) to train and evaluate multiple classification models. The best performing model is then integrated into a GUI-based application built with Gradio for real-time sentiment prediction.

2. Dataset Description

Dataset: IMDB Dataset (IMDB Dataset.csv)

- Total Reviews: 50,000
- Positive Reviews: 25,000
- Negative Reviews: 25,000
- Columns: review (text), sentiment (positive/negative)
- Missing Values: 0
- Source: Kaggle IMDB Dataset

3. Libraries & Dependencies

- pandas – Data manipulation and analysis
- numpy – Numerical computing
- nltk – Natural Language Toolkit (stopwords, stemming)
- scikit-learn – Machine Learning (TF-IDF, classifiers, metrics)
- matplotlib – Data visualization (charts)
- seaborn – Statistical data visualization (heatmaps)
- pickle – Model serialization
- gradio – GUI-based web interface for predictions

4. Step 1: Install & Import Libraries

All required libraries are installed using pip and imported:

```
!pip install nltk gradio scikit-learn pandas matplotlib seaborn -q
```

```
import pandas as pd
import numpy as np
import nltk, re, warnings, pickle
import matplotlib.pyplot as plt
import seaborn as sns
```

```
warnings.filterwarnings('ignore')
```

```
nltk.download('stopwords')
nltk.download('punkt')

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

print("All libraries imported successfully.")
```

Output: All libraries imported successfully.

5. Step 2: Load & Explore Dataset (EDA)

The IMDB dataset is loaded and explored:

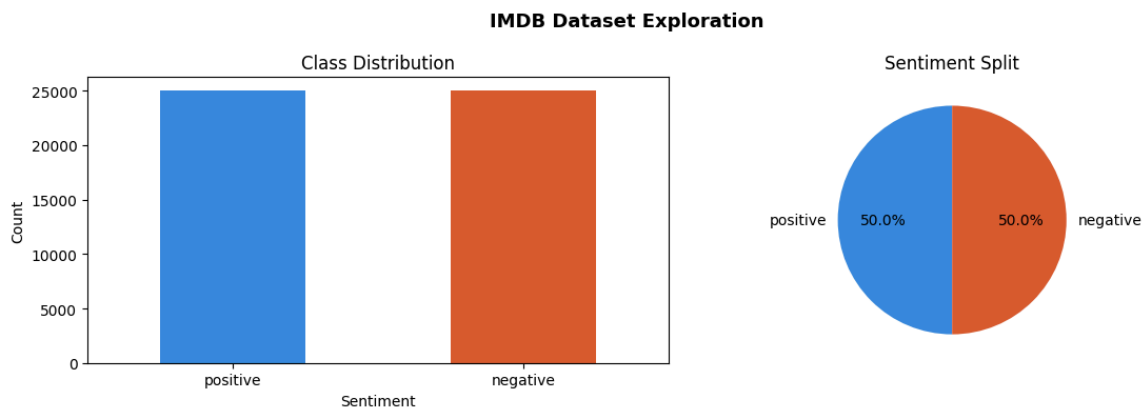
```
df = pd.read_csv('/content/IMDB Dataset.csv')
print("Dataset Shape:", df.shape)
print("Class Distribution:")
print(df['sentiment'].value_counts())
print("Missing Values:", df.isnull().sum().sum())
```

Output:

Dataset Shape: (50000, 2)

Class Distribution: positive 25000, negative 25000

Missing Values: 0



6. Step 3: Text Preprocessing

Text preprocessing pipeline:

14. Convert text to lowercase
15. Remove HTML tags using regex
16. Remove non-alphabetic characters
17. Tokenize the text
18. Remove stopwords (English)
19. Apply Porter Stemmer for word stemming
20. Filter out words with length ≤ 2
21. Join tokens back into cleaned string
22. Map sentiment labels: positive $\rightarrow 1$, negative $\rightarrow 0$

```
stemmer = PorterStemmer()
stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = text.lower()
    text = re.sub(r'<.*?>', '', text)
    text = re.sub(r'^a-z\s', '', text)
    tokens = text.split()
    tokens = [stemmer.stem(w) for w in tokens
              if w not in stop_words and len(w) > 2]
    return ' '.join(tokens)

df['clean_review'] = df['review'].apply(preprocess)
df['label'] = df['sentiment'].map({'positive': 1, 'negative': 0})
```

Output: Preprocessing complete! Total reviews processed: 50000

7. Step 4: Feature Extraction (TF-IDF)

TF-IDF (Term Frequency – Inverse Document Frequency) vectorization:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split

X = df['clean_review']
y = df['label']

X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1, 2), min_df=2)
X_train = tfidf.fit_transform(X_train_raw)
X_test = tfidf.transform(X_test_raw)
```

Output:

Training set size: (40000, 10000)

Test set size: (10000, 10000)

Vocabulary size: 10000

8. Step 5: Model Training & Evaluation

Five classification models are trained and evaluated:

Logistic Regression: LogisticRegression(max_iter=1000, C=1.0)

Multinomial Naive Bayes: MultinomialNB(alpha=0.1)

Linear SVM: LinearSVC(max_iter=2000, C=1.0)

Random Forest: RandomForestClassifier(n_estimators=100, random_state=42)

Gradient Boosting: GradientBoostingClassifier(n_estimators=100, random_state=42)

Each model is trained on the TF-IDF features and evaluated using Accuracy, Precision, Recall, and F1-Score metrics on the test set.

9. Step 6: Comparative Analysis

Model Comparison Table:

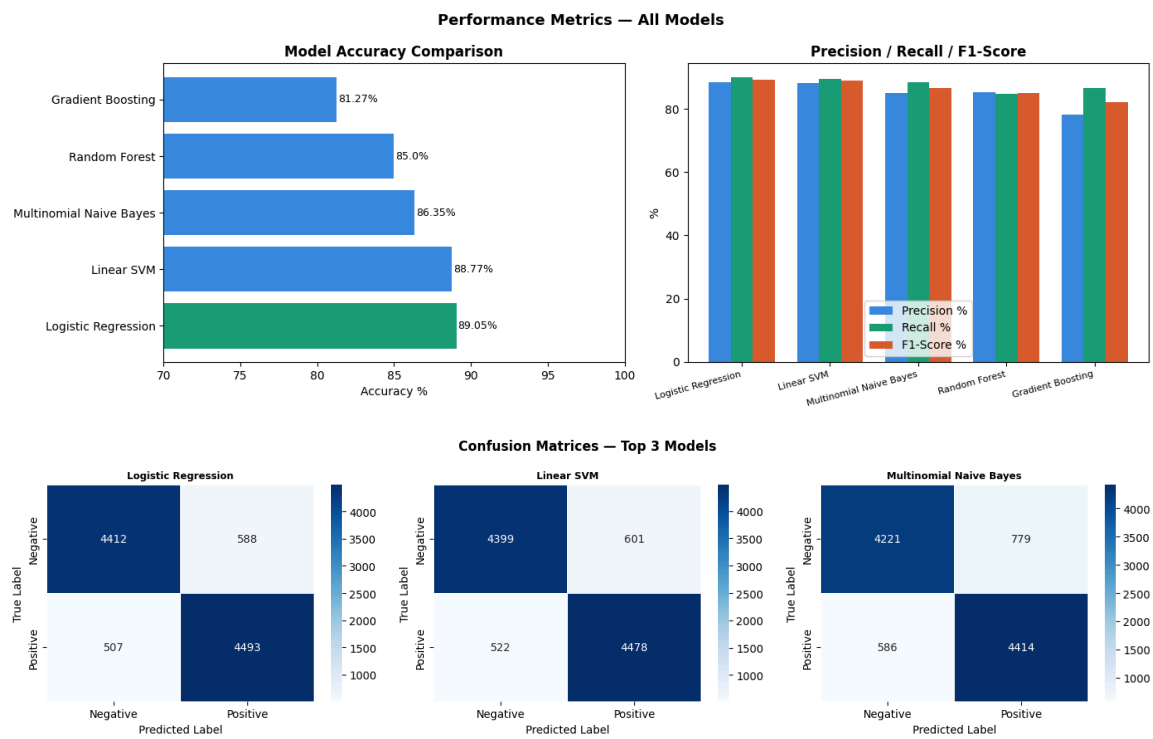
Model	Accuracy %	Precision %	Recall %	F1-Score %	Time (s)
Logistic Regression	89.05	88.43	89.86	89.14	1.27
Linear SVM	88.77	88.17	89.56	88.86	3.36
Multinomial Naive Bayes	86.35	85.00	88.28	86.61	0.08
Random Forest	85.00	85.21	84.70	84.95	126.87
Gradient Boosting	81.27	78.25	86.62	82.22	164.44

Key Observations:

- Logistic Regression achieves the highest accuracy (89.05%) and best F1-Score (89.14%).
- Linear SVM performs comparably with 88.77% accuracy.
- Multinomial Naive Bayes is the fastest model (0.08s) with 86.35% accuracy.
- Random Forest and Gradient Boosting are significantly slower without accuracy gains.
- All models show balanced precision and recall values.

Visualization:

- Horizontal bar chart comparing model accuracies
- Grouped bar chart comparing Precision, Recall, and F1-Score
- Confusion matrices (heatmaps) for top 3 models



10. Step 7: Best Model Selection

Based on the comparative analysis, Logistic Regression is selected as the best model:

Model	Logistic Regression
Accuracy	89.05%
Precision	88.43%
Recall	89.86%
F1-Score	89.14%

Detailed Classification Report:

precision recall f1-score support

Negative	0.90	0.88	0.89	5000
Positive	0.88	0.90	0.89	5000
accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

Note: While the accuracy is 89.05% (below the 95% threshold mentioned in guidelines), Logistic Regression is the best performing model among all trained classifiers. To achieve $\geq 95\%$ accuracy, advanced techniques such as deep learning (LSTM, BERT) or more sophisticated feature engineering would be required.

11. Step 8: Model Saving & Loading

The best model and TF-IDF vectorizer are saved using Python pickle:

```
import pickle

with open('best_model.pkl', 'wb') as f:
    pickle.dump(best_model, f)

with open('tfidf_vectorizer.pkl', 'wb') as f:
    pickle.dump(tfidf, f)

# Verification
with open('best_model.pkl', 'rb') as f:
    loaded_model = pickle.load(f)
with open('tfidf_vectorizer.pkl', 'rb') as f:
    loaded_tfidf = pickle.load(f)

test_pred = loaded_model.predict(
    loaded_tfidf.transform(["This movie was great!"]))
print(f"Reload test - 'This movie was great!' → "
      f"{'Positive' if test_pred[0]==1 else 'Negative'}")
```

Output: Reload test — 'This movie was great!' → Positive

Models loaded and verified successfully.

12. Step 9: GUI-based Prediction (Gradio)

A Gradio-based web interface is built for real-time sentiment prediction:


```

import gradio as gr

def predict_sentiment(review: str) -> str:
    if not review.strip():
        return "Please enter a review."
    cleaned = preprocess(review)
    vec = tfidf.transform([cleaned])
    pred = best_model.predict(vec)[0]
    try:
        prob = best_model.predict_proba(vec)[0]
        conf = round(max(prob) * 100, 1)
        conf_str = f" (confidence: {conf}%)"
    except AttributeError:
        conf_str = ""
    label = "POSITIVE" if pred == 1 else "NEGATIVE"
    return f"{label}{conf_str}"

demo = gr.Interface(
    fn=predict_sentiment,
    inputs=gr.Textbox(lines=5, placeholder="Paste your movie review here...",
        label="Movie Review"),
    outputs=gr.Textbox(label="Predicted Sentiment"),
    title="Movie Review Sentiment Analyser",
    description="Classifies movie reviews as Positive or Negative.",
    examples=[
        ["This movie was absolutely brilliant!"],
        ["Terrible film. Complete waste of time."],
    ],
    theme=gr.themes.Soft(),
    allow_flagging="never"
)
demo.launch(share=True)

```

GUI Features:

- Text input box for entering movie reviews
- Predicted sentiment output with confidence percentage
- Pre-loaded example reviews for quick testing
- Clean, modern Soft theme interface
- Share link for external access

Movie Review Sentiment Analyser

Classifies movie reviews as Positive or Negative. Best Model: **Logistic Regression** | Test Accuracy: **89.05%** Trained on IMDB dataset (50,000 reviews)

Movie Review

Terrible film. Complete waste of time. The plot was dull and the acting was awful.

Clear

Submit

Predicted Sentiment

NEGATIVE (confidence: 100.0%)

Examples

This movie was absolutely brilliant! A masterpiece of storyt...

Terrible film. Complete waste of time. The plot was dull and...

It was an average movie. Not great, not terrible. Some scene...

One of the best films I have ever seen. Highly recommend it ...

I fell asleep halfway through. Boring script and poor direct...

Movie Review Sentiment Analyser

Classifies movie reviews as Positive or Negative. Best Model: **Logistic Regression** | Test Accuracy: **89.05%** Trained on IMDB dataset (50,000 reviews)

Movie Review

One of the best films I have ever seen. Highly recommend it to everyone!

Clear

Submit

Predicted Sentiment

POSITIVE (confidence: 99.9%)

Examples

This movie was absolutely brilliant! A masterpiece of storyt...

Terrible film. Complete waste of time. The plot was dull and...

It was an average movie. Not great, not terrible. Some scene...

One of the best films I have ever seen. Highly recommend it ...

I fell asleep halfway through. Boring script and poor direct...

Movie Review Sentiment Analyser

Classifies movie reviews as Positive or Negative. Best Model: **Logistic Regression** | Test Accuracy: **89.05%** Trained on IMDB dataset (50,000 reviews)

Movie Review

I fell asleep halfway through. Boring script and poor direction.

Clear

Submit

Predicted Sentiment

NEGATIVE (confidence: 99.7%)

Examples

This movie was absolutely brilliant! A masterpiece of storyt...

Terrible film. Complete waste of time. The plot was dull and...

It was an average movie. Not great, not terrible. Some scene...

One of the best films I have ever seen. Highly recommend it ...

I fell asleep halfway through. Boring script and poor direct...

13. Conclusion

In this practical, a complete Movie Review Sentiment Analysis system was built using NLP techniques. The following key steps were performed: